APPENDIX A
PROOF OF THE STRONGEST-POSTCONDITION GUARANTEE

This appendix proves Proposition 1.

Proof. Denote $SP(\mathcal{F}, P, p_{start}, p_{end})$ by Φ. Let s_{entry} denote the function-entry state and s an endpoint valuation. We first establish that Φ characterizes the reachable values on each path's tracked locations, then prove maximality within the scope of the definition.

(Soundness): Consider a concrete execution represented by the incoming symbolic state under P. By (A1), it is well-defined and reaches p_{end} along a feasible path π. By (A2), symbolic execution enumerates this path, PC_π holds, and the symbolic store gives the concrete endpoint value of every tracked location. Thus, the reached valuation satisfies $\ell^{p_{end}} = \sigma_{p_{end},\pi}(\ell)$ for every $\ell \in \mathrm{dom}(\sigma_{p_{end},\pi})$. It therefore satisfies $PC_\pi \wedge \rho(\sigma_{p_{end},\pi})$ and hence Φ.

(Completeness): Let (s_{entry}, s) satisfy Φ. Some disjunct $PC_\pi \wedge \rho(\sigma_{p_{end},\pi})$ holds under this pair. Since PC_π includes P, the function-entry values satisfy the incoming constraints. Incoming-state exactness in (A2) supplies a corresponding segment-start state. By (A1) and the exact path conditions and symbolic stores in (A2), concrete execution follows π to an endpoint valuation t that agrees with s at every location tracked on that path. Thus all tracked values specified by the disjunct are realized by a concrete execution.

(Maximality): Let Ψ be any postcondition within this scope such that $\{P\}\,\mathcal{F}\,\{\Psi\}$ holds. For any (s_{entry}, s) satisfying Φ, completeness supplies a concrete endpoint valuation t on a feasible path π. The Hoare triple gives $\Psi(s_{\text{entry}}, t)$. On π, the truth of Ψ depends only on the function-entry values and the tracked endpoint values. Since s and t agree on these tracked values and share the same function-entry state, $\Psi(s_{\text{entry}}, s)$ also holds. Therefore $\Phi \Rightarrow \Psi$. Under the exact path conditions, Φ itself depends only on the function-entry values and the corresponding path's tracked endpoint values, so it lies within the same scope. Together with soundness, this establishes the strongest-postcondition claim. □

APPENDIX B
DEFAULT SEPARATION AND POINTER ALIASING

QCP needs initial memory predicates that justify dereferences. The assertion `Require emp` describes an empty heap, so symbolic execution of the swap below stops at the first read of `*px`: it cannot derive the precondition of that memory read. A single specification can cover both pointer layouts by stating them as alternatives:

Swap with aliased or disjoint pointers

```
void swap(int *px, int *py)
/*@ With x y
    Require (px == py && x == *px) ||
            (px != py && x == *px && y == *py)
    Ensure  (px == py && x == *px) ||
            (px != py && y == *px && x == *py)
*/
{
    int t = *px;
    *px = *py;
    *py = t;
}
```

In the alias branch, `px == py` and one points-to predicate describe a shared cell. In the other branch, QCP translates the two dereferences into points-to predicates joined by separating conjunction ($*$), which describes disjoint cells. Symbolic execution produces a return proof obligation for each branch; these obligations still require proof. Because a Frama-C/WP contract without separation leaves aliasing unconstrained, representing its valid overlapping and disjoint inputs in QCP would require explicit heap alternatives, so the default precondition generator selects only the disjoint branch to limit symbolic-execution branching. An aliasing input can be supported by replacing the default separation precondition with an appropriate aliasing predicate, as in the first branch above.

APPENDIX C
LLM FALLBACK DETAILS

In the fallback, the LLM acts as a semantic summarization component for the missing symbolic-execution step. It receives the current code fragment, available symbolic states and path conditions, previously generated callee specifications or loop summaries, and an ACSL skeleton. The LLM is tasked with following the relevant paths, tracking changes to variables and memory locations, and proposing ACSL clauses that summarize the observed behavior. The resulting candidates must still be proved by Frama-C/WP.

We use this fallback for four function-level tasks. *Precondition generation* asks the LLM to infer candidate `requires` clauses describing the validity, separation and bound constraints. *Assigns-clause generation* asks the LLM to infer the function's footprint, i.e., `assigns` clauses describing which visible memory locations may be modified. *Postcondition generation* asks the LLM to summarize the relation between the pre-state and the post-state, filling candidate `ensures` clauses. *Inductive-predicate generation* asks the LLM to decide whether the program requires an inductive predicate and, when needed, to propose one for capturing inductive relations in recursive data structures or recursive functions.

APPENDIX D
LOOP ANALYSIS PROCEDURE

Algorithm 2: Loop Analysis LoopAnalysis

Input: loop body $\mathcal{F}_{loop}$, precondition *Pre*, loop stack *lS*, call stack *cS*, annotated functions *aF*, LLM
Output: annotated $\mathcal{F}_{loop}$, sets of unchanged variables *uV* and non-inductive variables *nV*

1 $\mathcal{F}_{unfolded} \leftarrow \text{UnfoldLoop}(\mathcal{F}_{loop})$;
2 $P \leftarrow Pre$;
3 $p_{start} \leftarrow p_{unfolded_0}$;
4 $p \leftarrow \text{SEStart}(\mathcal{F}_{unfolded}, P, p_{start})$;
5 **while** $p \neq p_{unfolded_-1}$ **do**
6 $P \leftarrow SP(\mathcal{F}_{unfolded}, P, p_{start}, p)$; `// Same as FuncSpec`
7 $p_{start} \leftarrow p$;
8 **if** $p = p_{nestedLoopEntry}$ **then**
9 $lS.\text{push}(\mathcal{F}_{nested})$;
10 **if** $p = p_{nestedLoopExit}$ **then**
11 $\mathcal{F}_{nested} \leftarrow lS.\text{pop}()$;
12 $\mathcal{F}_{loop} \leftarrow \text{NestedLoopLLMCall}(\mathcal{F}_{loop}, \mathcal{F}_{nested}, LLM)$;
13 **if** $p = p_{callee}$ **then**
14 $\mathcal{F}_{callee} \leftarrow \text{FuncSpec}(\mathcal{F}_{callee}, cS, aF)$;
15 $p \leftarrow \text{SEStart}(\mathcal{F}_{unfolded}, P, p_{start})$;
16 ...; `// Same as FuncSpec, construct the final postcondition` $\mathcal{S}$
17 $uV \leftarrow \text{FetchUnchangedVars}(\mathcal{S})$;
18 $nV \leftarrow \text{FetchNonInductiveVars}(\mathcal{S})$;
19 **return** $\mathcal{F}_{loop}, uV, nV$;

LoopAnalysis (Algorithm 2) symbolically executes a single-iteration unfolding $\mathcal{F}_{unfolded}$ of the loop body (Lines 1–4) and then advances through it one program point at a time (Lines 5–15), exposing the update and frame facts of one iteration. The key difference from FuncSpec lies in the handling of nested loops: whereas the outer loop uses symbolic-execution-derived preconditions and templates to guide LLM synthesis (OuterLoopLLMCall), an inner loop is processed purely through NestedLoopLLMCall (Lines 10–12), which infers its invariant directly from the code context without preconditions or templates, because a well-defined precondition is not available at the enclosing loop boundary. Nested loops are thus discharged bottom-up using the loop stack *lS*, while function calls are handled consistently with FuncSpec through the call stack *cS* (Lines 13–14): the callee is summarized before symbolic execution of the unfolded body resumes. Once the unfolded body has been fully executed, its final postcondition is constructed and all nested loops are annotated with the invariants generated for them; this postcondition is then used to classify the unchanged variables *uV* and non-inductive variables *nV* of the outer loop (Lines 16–18).

The driving LoopInvGen procedure (Algorithm 3) wraps LoopAnalysis with template generation (LoopInvTemGen), a single OuterLoopLLMCall, and a verification-and-refine pass.

Algorithm 3: Loop Invariant Generation LoopInvGen

Input: function $\mathcal{F}$, loop entry p_{loop}, loop stack *lS*, call stack *cS*, annotated functions *aF*, loop precondition P, LLM
Output: function $\mathcal{F}$ annotated with loop invariant $\mathcal{I}$ for the loop $\mathcal{F}_{loop}$ at p_{loop}

`//` $\mathcal{F}_{loop}$ `is the loop body at` p_{loop} `in` $\mathcal{F}$
1 $\text{ThinkInNaturalLanguage}(\mathcal{F}_{loop}, LLM)$;
2 $lS.\text{push}(\mathcal{F}_{loop})$;
3 $\mathcal{F}_{loop}, uV, nV \leftarrow \text{LoopAnalysis}(\mathcal{F}_{loop}, P, lS, cS, aF, LLM)$;
4 $lS.\text{pop}()$; `// lS depth is unchanged across this call`
5 $\mathcal{T} \leftarrow \text{LoopInvTemGen}(\mathcal{F}_{loop}, P, uV, nV)$;
6 $\mathcal{I} \leftarrow \text{OuterLoopLLMCall}(\mathcal{F}, \mathcal{F}_{loop}, P, \mathcal{T}, LLM)$;
7 $\mathcal{F} \leftarrow (\mathcal{F}, \mathcal{I})$;
8 **if** $\neg\text{Verify}(\mathcal{F})$ **then**
9 $\mathcal{F} \leftarrow \text{Refine}(\mathcal{F}, \mathcal{I}, P, loop, LLM)$;
10 **return** $\mathcal{F}$;

APPENDIX E
SYSTEM DESIGN AND PROMPT CONSTRUCTION

A. Execution and Verification Backbone

The implementation combines three off-the-shelf components: the QCP symbolic execution engine [17], [18], an LLM, and the Frama-C/WP automated verifier [19], and orchestrates them through a thin translation layer. Symbolic execution drives the pipeline and pauses at each loop and function call, where the LLM proposes the missing annotations from the exposed symbolic state and Frama-C/WP discharges the resulting proof obligations.

The choice of QCP and Frama-C is motivated by their complementary strengths. QCP is used as the symbolic execution engine because it produces Coq-verified postconditions and, more importantly, exposes its symbolic state (path conditions plus symbolic stores) at arbitrary program points in a logical form that is directly reusable for specification synthesis. Crucially, QCP satisfies the symbolic-execution conditions of Proposition 1, thereby providing the foundation for the strongest-postcondition guarantee on loop-free segments. Other widely used alternatives such as Klee [39] are not built on Hoare logic and accordingly neither expose intermediate states at arbitrary locations nor present them in a logic-oriented form suitable for specification synthesis. QCP also has strong support for programs that manipulate complex data structures and ships with a library of verified inductive predicates and lemmas, which matches our goal of producing formally precise specifications. The gap QCP leaves — it requires user-supplied loop invariants and callee pre-/postconditions, and it may emit lemmas that need additional Coq interactive proofs — is exactly the gap filled by the other two components. The LLM proposes the missing invariants and specifications from the symbolic state QCP exposes; Frama-C/WP then discharges the resulting proof obligations automatically through SMT under the LLM-friendly ACSL surface syntax. The three components therefore split the work cleanly: QCP supplies the symbolic semantics, the LLM supplies the specification candidates, and Frama-C/WP closes the verification loop.

Stitching these three components together requires a translation layer, because QCP, the LLM, and Frama-C/WP each speak a different annotation language: QCPL (native to QCP), ACSL (native to Frama-C/WP), and natural-language prompts (for the LLM). The translator maintains a bidirectional bridge between ACSL and QCPL: it translates ACSL clauses into QCPL so that QCP can use them across loop and function boundaries, and it converts QCP's symbolic-state output into

ACSL skeletons (paired with natural-language guidance for the LLM) and ACSL clauses (for Frama-C/WP verification). A deliberate design choice is that the LLM never sees QCPL; it only operates on ACSL plus natural-language guidance derived from the symbolic state. This avoids the common failure mode of confusing two formally similar but semantically distinct languages, and the skeleton-plus-prose discipline ensures structural well-formedness before the LLM fills in the logical content. When verification fails, error information is translated back into natural language, together with its context, and routed into the next LLM call—closing the loop without exposing the verifier's internal representation to the model either.

B. Prompt Construction

Every LLM call in SESPEC follows a fixed template, but the concrete prompt is specialized to the input program. A lightweight static classifier inspects the source code and assigns each program to one of five routes: recursive data structure, recursive program, array, numeric, and plain, where plain is the default route for programs that do not need helper definitions. The selected route determines which worked examples, category-specific ACSL guidance, and route-specific rule fragments are inserted on top of the universal ACSL pitfalls block. Thus, the same template yields substantively different prompts for, say, a numeric recurrence and a linked-list cursor.

This routing drives the five LLM call sites in the pipeline: loop invariant generation, precondition generation, postcondition/assigns generation, syntax repair, and verifier-guided semantic refinement. In addition, the symbolic executor supplies the state-dependent content of each prompt — the variable state and the path constraints — together with an annotation skeleton to be completed by the LLM. At refinement time, the verifier's typed error categories are additionally folded into the prompt. All prompt templates are listed in Section E-C.

C. Full Prompt Templates

This section lists the templates used by the main LLM call sites referenced in Section E-B, abridged to their static skeleton and runtime slots (the verbose in-prompt admonitions are elided). Each template's static body is shown in black; runtime-injected slots appear in gray with colored tags from the shared figure palette: LOOP/SYMBOLIC for loop-aware symbolic context, ROUTE/CALL for route and call-site context, and VERIFIER/SPECIFICATION for verifier errors and specification edits.

1) Loop-invariant generation: The example below is a filled loop-invariant prompt for a small branch-then-loop program, with the long example and guidance blocks elided.

Loop-invariant generation prompt

```
### Role / Task ###
Given a C program with a loop, generate ACSL loop invariants
that let Frama-C verify it.

ROUTE  examples + ACSL helper policy for the matched route, omitted
GUIDANCE  strength / supplementary / assertion guides, omitted

### Rules ###
- Only fill the loop-invariant PLACE_HOLDERs (or add a helper block
  before the function); keep existing annotations unchanged.
- Use the pre-condition, not \at(var, LoopEntry).
- Emit one self-contained ```c``` block.

SYMBOLIC STATE  supplied by the symbolic executor
Pre-condition:
    (\at(a,Pre)==0 && y==0 && x==1 && a==\at(a,Pre))
 || (\at(a,Pre)!=0 && y==0 && x==0 && a==\at(a,Pre))

Loop + skeleton to complete:
/*@
  loop invariant (\at(a,Pre)!=0) ==> PLACE_HOLDER_x;
  loop invariant (\at(a,Pre)!=0) ==> (a==\at(a,Pre));
  ... (symmetric a==0 branch omitted) ...
*/
while (y < 100000) { x = x + y; y = y + 1; }
```

2) Precondition fallback:

Precondition fallback prompt

```
### Role: ###
Generate ONLY function-level preconditions plus supporting
predicate/logic declarations needed by them.

### Goal: ###
Rule out Frama-C runtime errors:
- pointer dereferences must be covered by \valid or
  \valid_read;
- array ranges must cover all indexed accesses;
- inductive data structures use structural predicates from
  examples.

ROUTE EXAMPLES
{examples}

### Output format: ###
Output exactly one fenced ```acsl``` block:
/*@ predicate <name>(...) = ...; */
/*@ logic <type> <name>(...) = ...; */
/*@
  requires <clause>;
*/

### Hard rules: ###
- Output only ACSL annotations.
- Produce requires only: no ensures, assigns, or loop invariant.
- Each predicate or logic declaration lives in its own /*@ ... */
↪ block.

Function under specification:
{code}
```

3) Postcondition and assignment generation:

Postcondition / assignment generation prompt

```
### Task: ###
Transform the following C code into an ACSL-annotated version.

### Rules: ###
- Only fill ensures PLACE_HOLDER and assigns PLACE_HOLDER.
- The input is a complete C file; callee specifications are
  read-only context.
- Output only the modified target function block. The
  harness splices it back into the original file.
- Helper predicate/logic definitions are allowed and required
  if the target specification references a name not already defined.
- Replace the ensures PLACE_HOLDER with as many strong,
↪ non-redundant
  clauses as the behavior admits (typically 3-8): the closed-form
  result, per-branch facts, and frame consequences.
- Inline relations directly in ensures; add a predicate/logic helper
  only when the relation is recursive or higher-order.

LOOP-AWARE DERIVATION
{loop_guide}

### ACSL pitfalls ###
UNIVERSAL + SYMBOLIC-ROUTE HINTS
{category_hints}

RESOLVED ARRAY SIZES
{array_length_facts}

### C Program Code: ###
{code}
```

4) Syntax repair: A deterministic ACSL fixer (Section IV-C) runs before this prompt and clears the common mechanical faults; the template below is the LLM fallback used only when the fixer cannot resolve the parser error.

Syntax-repair prompt

```
### Role: ###
You are an expert in C and Frama-C.

### Task: ###
Correct syntactically incorrect ACSL annotations using
Frama-C errors.

Syntax error:
VERIFIER ERROR
{error_str}

C code with incorrect ACSL annotations:
{c_code}

### ACSL pitfalls ###
UNIVERSAL + SYMBOLIC-ROUTE HINTS
{category_hints}

### Rules: ###
- Do not modify function bodies or signatures.
- Only change ACSL annotations.
- New top-level predicate/logic blocks may be added before
  the function.
- If an unbound logic/predicate name is reported, define it
  or remove every clause that references it.
```

5) Verifier-guided semantic refinement: Fig. 6 gives the refinement prompt used by the verifier-guided refinement loop of Section IV-C.

Verifier-guided refinement prompt

```
### Task: ###
Frama-C reports verification failures grouped by category. Each
↪ category
has a different fix strategy; only relevant strategies are shown.

### Categorized errors ###
ACTIVE ERROR BUCKETS
{error_str}

### Per-category fix strategy ###
ACTIVE STRATEGIES ONLY
{strategy_blocks}

### Current C code with annotations ###
{c_code}

LOOP-AWARE DERIVATION
{loop_guide}

### ACSL pitfalls ###
UNIVERSAL + SYMBOLIC-ROUTE HINTS
{category_hints}

### Output rules ###
- The target function's requires may be strengthened when necessary.
EDIT MODE
{output_format_block}
- Do not modify user assert statements.
- Do not modify correct loop invariants or correct postconditions.
- Wrap the corrected target block, or the complete file in
↪ full-file mode,
  in one ```c``` fenced block.
```

Fig. 6. Verifier-guided semantic refinement prompt.

Appendix F
Refinement Procedure and Budget Ablation

Algorithm 4 gives the verifier-guided refinement procedure of Section IV-C, and Table VII gives the category-to-strategy mapping used by its strategy-driven layer. Fig. 7 then isolates the effect of refinement budget on gpt-5.4-mini. The cumulative *Valid* rate rises from 24.2% at $k{=}0$ to 60.2% at $k{=}3$, 73.0% at $k{=}5$, and 79.2% at $k{=}10$, and then reaches 93.3% after the final Houdini pass. The marginal gain after $k{=}5$ is smaller, while Houdini contributes a final $+14.1$ pp by pruning failing clauses from the best refinement snapshots. In contrast, the *trivial refine* ablation, which uses only a plain error-plus-code prompt, reaches 70.0% under the same 10-iteration budget and final Houdini pass. This shows that differential refinement prompts and best rollback together account for roughly 23 pp of additional validity.

Appendix G
Full Pass@k Benchmark Results

Table VIII supplements Table II (Section V) with the complete Pass@1/3/5 results per benchmark and model.

Appendix H
Translation Detail

We used GPT-5.4 to translate the 250 FormalBench and 59 SpecGenBench Java programs into C; the other 91 SV-COMP cases are native C. For each Java case, the model was asked to preserve the target method's branches, loops, assignments, and returns while adapting its types and memory operations to C. We removed JML annotations and goal assertions and renamed the target function to `foo` followed by its case ID. Programs that could not be translated into the supported C fragment and compiled were excluded; for example, SpecGenBench's `ConvertToTitle` uses Java `StringBuffer.append`

Algorithm 4: Verifier-Guided Refinement Refine

Input: $\mathcal{F}$ with candidate annotation $\mathcal{A}_0$, symbolic context Γ, target kind $k \in \{loop, spec\}$, *LLM*
Output: $\mathcal{F}^*$ with verifier-checked annotation $\mathcal{A}^*$

$\mathcal{A} \leftarrow \mathcal{A}_0$;
$\mathcal{F}^- \leftarrow \mathcal{F} - \mathcal{A}_0$;
while *Errors* $\leftarrow \neg$Verify$(\mathcal{F})$ **do**
  if *reach iteration limit* **then**
    `// Layer 3: Houdini-style pruning`
    $\mathcal{A} \leftarrow$ Houdini$(\mathcal{F}^-, \mathcal{A}, Errors)$;
    $\mathcal{F} \leftarrow (\mathcal{F}^-, \mathcal{A})$;
    break;
  if *Errors* $=$ *syntax error* **then**
    `// Layer 1: syntax repair`
    $\mathcal{A} \leftarrow$ RepairCallLLM$(\mathcal{A}, \Gamma, Errors)$;
  else
    `// Layer 2: strategy-driven refinement or regeneration`
    guidance $\leftarrow$ BuildGuide(Classify$(Errors, k), \Gamma)$;
    $\mathcal{A} \leftarrow$ RefineCallLLM$(\mathcal{A}, \Gamma,$ *guidance*$)$;
  $\mathcal{F} \leftarrow (\mathcal{F}^-, \mathcal{A})$;
return $\mathcal{F}$;

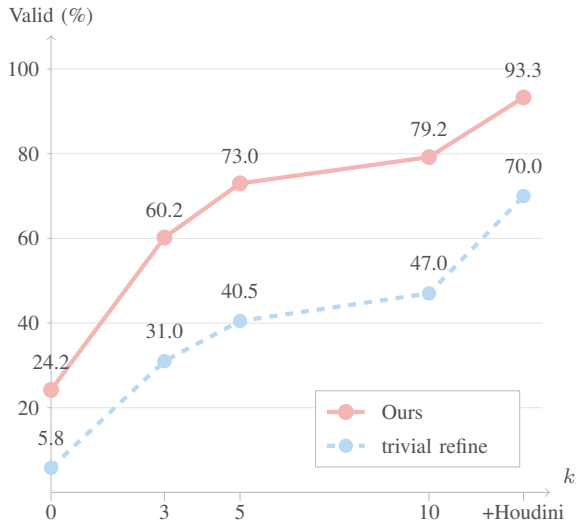


Fig. 7. Cumulative *Valid* rate on gpt-5.4-mini over 400 cases as the refinement budget k increases, for the full SESPEC pipeline (*Ours*) versus a *trivial refine* baseline.

and `reverse` and was not included. All 400 selected C programs pass GCC C11 compilation: 309 translated programs and 91 native-C programs.

For the SPECGEN evaluation, we use the original Java versions of the 309 Java-origin programs and translate the 91 native-C SV-COMP programs into Java. Thus, SPECGEN is evaluated on all 400 corresponding programs, and its validity rates use the same denominator of 400.

In the Java-to-C translations, Java `int`, `long`, and `boolean` are represented by C `int`, `long long`, and 0/1-valued `int`. An array parameter `int[] a` becomes `int *a, int a_len`; `a.length` becomes `a_len`, and array lengths are passed through calls. A local `new int[n]` becomes a C allocation with a stored length. Constants such as `Integer.MIN_VALUE` and the few string operations are adapted to their C counterparts.

Compilation checks syntax, not full semantic equivalence. Cross-language comparisons are restricted to normal executions with valid memory accesses, successful allocation, and no signed overflow.

We also checked translation behavior by differential execution on the 309 program pairs translated from Java to C. The check covered 4,325 comparable executions using deterministic edge-case and seeded pseudorandom inputs, and compared the return value and any modified integer-array arguments. The Java programs were compiled for Java 8, and the C programs were compiled with GCC C11 and undefined-behavior checks. Of these executions, 4,323 produced matching observations; the other two triggered C undefined behavior in the same program. Thus, 308 of 309 pairs showed no discrepancy on their tested inputs, while one pair exposed a cross-language semantic difference.

In that pair, case 112, input `'0'` makes the expression `1 << (c - 'a')` shift by a negative count in C, whereas Java defines the shift and returns 1. This discrepancy is outside the common defined-behavior domain, but it shows why compilation and finite-input agreement cannot establish unrestricted Java/C equivalence.

APPENDIX I
SPECIFICATION STRENGTH COMPARISON

This appendix collects the complete material for the dimension-wise specification-strength study of Section VI-C: the comparison protocol, the judge configuration and prompts, the full pairwise results for both baselines including the cross-language comparison against SPECGEN (Java/JML), and the A/B-order audit of the LLM judge.

A. Comparison protocol

Validity certifies that a specification is discharged by its verifier, but it does not rank how informative it is. We therefore compare accepted input domains (preconditions), normal-return guarantees (postconditions), and predicates at corresponding loop points (loop invariants) independently. Each dimension uses a separate judge prompt and API call. Preconditions are classified by set inclusion between their accepted input domains; postconditions are compared by bidirectional implication over the common input domain; and invariants are compared at corresponding loop points. Thus, the two directional columns in Table IX mean more permissive for preconditions and logically stronger for postconditions and invariants. Fig. 3 also combines the precondition and postcondition verdicts into a contract comparison. A contract is stronger when its accepted input domain is no smaller and its postcondition is no weaker, with at least one strict advantage. Equality requires equality in both dimensions; conflicting or unresolved verdicts enter Other. Loop invariants remain separate. A missing precondition or postcondition denotes `true`. Expert review corrected 32 judge verdicts, shown as transitions in Table IX; any implication that cannot be established is included in Incomp.

TABLE VII
REFINEMENT STRATEGIES BY VERIFIER-FAILURE CATEGORY.

Target	Verifier feedback	Refinement strategy
Loop invariant	Establishment (base) obligation fails	Drop or guard facts not implied by the pre-loop state.
	Preservation obligation fails	Rebuild the inductive relation from the loop body's one-step effect.
	Invariant holds but the downstream goal or assertion is unproved	Add loop-exit consequences linking the negated guard to the inductive and unchanged facts.
	Several invariants fail or the candidate is globally inconsistent	Discard the candidate and regenerate from the original template.
Function specification	Target postcondition obligation fails	Drop over-strong claims or split them into path-sensitive clauses.
	Callee-instance obligation fails at a call site	Strengthen caller-side facts before the call, or weaken an over-strong callee precondition.
	Postcondition holds but a user assertion still fails	Add the missing bridge facts to the specification or to the loop-exit facts.
	Frame (`assigns`) obligation fails	Add only the missing externally visible locations.

B. Judge configuration

We used GPT-5.4 with a 2,048-token output limit, no input truncation, and the provider-default temperature, top-p, and reasoning effort. Each precondition, postcondition, and loop-invariant comparison used a separate API call and the corresponding prompt in Section I-C. The expert review was conducted by two master's-level researchers with formal-verification backgrounds.

C. Pairwise specification strength judges

Fig. 8 gives the three independent judge prompts used for the pairwise comparisons reported in Section VI-C and Table IX. Each invocation receives only the target function's annotation blocks relevant to that dimension; the complete prompt text and structured responses are included in the artifact.

Precondition judge

```
[System]
Compare only accepted input domains. A missing
precondition means true. Report which predicate is
WEAKER, i.e., which one accepts more inputs.
Do not compare postconditions or invariants.

[User]
Evaluate independently:
  B_to_A: P_B implies P_A
  A_to_B: P_A implies P_B
B_to_A only => A_accepts_more
A_to_B only => B_accepts_more
both => equivalent
neither/unknown => incomparable
Return both directions, the relation, and a concise
implication-based reason as strict JSON.
```

Postcondition judge

```
[System]
Compare only normal-return guarantees on the common
accepted input domain. Missing preconditions or
postconditions mean true. Ignore invariant content.

[User]
Let D = P_A AND P_B. Evaluate independently:
  A_to_B: D AND Q_A implies Q_B
  B_to_A: D AND Q_B implies Q_A
A_to_B only => A_guarantees_more
B_to_A only => B_guarantees_more
both => equivalent
neither/unknown => incomparable
Do not reward a wider input domain or infer an
unstated guarantee from the body. Return both
implication directions, relation, and reason as JSON.
```

Loop-invariant judge

```
[System]
Compare only explicit loop invariants at corresponding
program points. Ignore function contracts.

[User]
At each point, conjoin that side's invariants.
Evaluate independently at every point:
  A_to_B: I_A implies I_B
  B_to_A: I_B implies I_A
A_to_B only => A_invariants_stronger
B_to_A only => B_invariants_stronger
both everywhere => equivalent
neither/unknown => incomparable
Do not merge different loop points or infer reachable
states from bodies. Return both implication directions,
relation, and a concise decisive reason as JSON.
```

Fig. 8. Independent strength-judge prompts used before expert review. A and B denote two anonymized specifications; tool identities are not disclosed to the judge.

D. Full pairwise results

Table IX reports the complete dimension and contract results on the both-valid sets for both baselines. The AUTOSPEC (C/ACSL) rows and the Frama-C/WP cross-check are discussed in Section VI-C; the remainder of this subsection analyzes the cross-language comparison against SPECGEN (Java/JML).

Against SPECGEN, SESPEC leads on loop invariants but SPECGEN leads on postconditions and combined contracts. One contributing factor is the difference between the two verification settings: the Java/JML and OpenJML stack supports high-level functional contracts without the pointer-validity and buffer-related obligations required by ACSL and Frama-C/WP. In the C setting, these additional memory-model obligations can make some postconditions difficult for Frama-C/WP to discharge, so SESPEC's verification-driven pipeline may retain fewer postconditions when the corresponding proof obligations cannot be discharged within the allotted budget. The comparison therefore reflects both specification generation and differences in the underlying language and verification stacks.

We omit a WP cross-check for SPECGEN because translating Java/JML reference and array semantics into ACSL would confound the comparison.

E. Position-bias audit (A/B order)

The primary run placed the baseline in A and SESPEC in B. To test order sensitivity, we repeated every judge decision with SESPEC in A and the baseline in B, then mapped the output

TABLE VIII
SPECIFICATION GENERATION PERFORMANCE UNDER PASS@1, PASS@3, AND PASS@5 (COMPLETE TABLE).

Benchmark	Model	Pass@1 (%)			Pass@3 (%)			Pass@5 (%)		
		Syn.	Val.	Acc.	Syn.	Val.	Acc.	Syn.	Val.	Acc.
SyGuS	GPT-4o-mini	90.98	87.97	78.20	97.74	95.49	88.72	98.50	96.99	90.23
	GPT-3.5-turbo	93.23	88.72	84.21	97.74	97.74	92.48	98.50	98.50	93.23
	GPT-4o	98.50	98.50	93.98	99.25	99.25	99.25	99.25	99.25	99.25
	Claude 3.7 Sonnet	100.00	100.00	99.25	100.00	100.00	99.25	100.00	100.00	99.25
OOPSLA	GPT-4o-mini	80.43	65.22	32.61	89.13	86.96	43.48	89.13	86.96	47.83
	GPT-3.5-turbo	82.61	54.35	41.30	86.96	65.22	45.65	86.96	67.39	47.83
	GPT-4o	91.30	69.57	60.87	95.65	76.09	73.91	100.00	82.61	76.09
	Claude 3.7 Sonnet	93.48	69.57	63.04	97.83	80.43	76.09	97.83	93.48	86.96
SV-COMP	GPT-4o-mini	61.90	61.90	28.57	80.95	80.95	42.86	85.71	80.95	57.14
	GPT-3.5-turbo	95.24	90.48	66.67	95.24	90.48	66.67	95.24	90.48	66.67
	GPT-4o	90.48	90.48	52.38	100.00	100.00	71.43	100.00	100.00	76.19
	Claude 3.7 Sonnet	100.00	100.00	71.43	100.00	100.00	71.43	100.00	100.00	76.19
NLA	GPT-4o-mini	90.00	83.33	76.67	90.00	83.33	76.67	90.00	83.33	76.67
	GPT-3.5-turbo	76.67	70.00	50.00	80.00	80.00	56.67	80.00	80.00	56.67
	GPT-4o	93.33	90.00	83.33	96.67	96.67	86.67	96.67	96.67	86.67
	Claude 3.7 Sonnet	96.67	96.67	86.67	96.67	96.67	86.67	96.67	96.67	86.67
numer-S	GPT-4o-mini	95.00	90.00	72.50	100.00	95.00	75.00	100.00	95.00	75.00
	GPT-3.5-turbo	92.50	85.00	62.50	95.00	90.00	65.00	95.00	92.50	67.50
	GPT-4o	100.00	100.00	82.50	100.00	100.00	82.50	100.00	100.00	82.50
	Claude 3.7 Sonnet	100.00	97.50	85.00	100.00	100.00	85.00	100.00	100.00	85.00
Frama-C	GPT-4o-mini	54.90	49.02	47.06	62.75	50.98	49.02	64.71	50.98	49.02
	GPT-3.5-turbo	76.47	60.78	52.94	82.35	62.75	54.9	82.35	62.75	54.9
	GPT-4o	82.35	72.55	66.67	86.27	76.47	70.59	86.27	78.43	72.55
	Claude 3.7 Sonnet	86.27	78.43	72.55	92.16	82.35	76.47	92.16	84.31	76.47
pIp	GPT-4o-mini	90.00	88.00	66.00	92.00	92.00	72.00	92.00	92.00	76.00
	GPT-3.5-turbo	84.00	76.00	66.00	88.00	78.00	72.00	88.00	78.00	76.00
	GPT-4o	94.00	78.00	64.00	94.00	78.00	76.00	94.00	78.00	78.00
	Claude 3.7 Sonnet	100.00	88.00	82.00	100.00	90.00	88.00	100.00	92.00	88.00
list-S	GPT-4o-mini	62.50	41.67	33.33	75.00	45.83	41.67	79.17	54.17	45.83
	GPT-3.5-turbo	70.83	20.83	8.33	79.17	25.00	16.67	79.17	25.00	16.67
	GPT-4o	70.83	62.50	54.17	87.50	83.33	75.00	91.67	87.50	79.17
	Claude 3.7 Sonnet	95.83	83.33	75.00	100.00	100.00	95.83	100.00	100.00	95.83

TABLE IX
PAIRWISE SPECIFICATION RELATIONS ON BOTH-VALID SETS, REPORTED AS LLM JUDGE → JUDGE + EXPERT REVIEW. INVARIANT COMPARISONS INCLUDE ONLY PAIRS FOR WHICH BOTH TOOLS PRODUCED LOOP INVARIANTS.

Baseline	Dimension	Evaluated	SESPEC side	Baseline side	Equal	Incomp.
AUTOSPEC (C/ACSL)	Preconditions	698	96→96	290→290	311→311	1→1
	Postconditions	698	342→342	18→18	320→320	18→18
	Contracts	698	172→172	64→64	200→200	262→262
	Loop invariants	540	120→120	50→52	5→5	365→363
SPECGEN (Java/JML)	Preconditions	673	161→164	178→174	221→222	113→113
	Postconditions	673	102→100	246→244	218→220	107→109
	Contracts	673	91→92	186→186	106→105	290→290
	Loop invariants	473	183→183	9→9	2→2	279→279

back to the original tool orientation. Table X compares the raw judge results before expert correction. The contract row combines the precondition and postcondition verdicts under the rule in Section VI-C. Pairs with no substantive predicate on either side are deterministically equal and are included in the reported agreement. The principal AutoSpec trends remain stable under the swapped order. For SpecGen, the postcondition and invariant directions persist, while the precondition direction reverses and the contract direction becomes a tie under the swapped order, indicating greater order sensitivity for this cross-language comparison.

TABLE X
A/B-ORDER AUDIT OF THE LLM JUDGE. OUTCOME TUPLES ARE SESPEC/BASELINE/EQUAL/INCOMPARABLE.

Dimension	Pairs	Agreement	Primary	Swapped
AUTOSPEC (C/ACSL)				
Preconditions	698	695 (99.6%)	96/290/311/1	99/288/310/1
Postconditions	698	672 (96.3%)	342/18/320/18	349/11/325/13
Contracts	698	675 (96.7%)	172/64/200/262	187/64/199/248
Loop invariants	540	435 (80.6%)	120/50/5/365	160/49/5/326
SPECGEN (Java/JML)				
Preconditions	673	546 (81.1%)	161/178/221/113	200/86/232/155
Postconditions	673	518 (77.0%)	102/246/218/107	141/203/217/112
Contracts	673	539 (80.1%)	91/186/106/290	129/129/110/305
Loop invariants	473	382 (80.8%)	183/9/2/279	198/3/1/271

Case 1: symbolic state exposes loop phases

```
int unknown();

/*@ requires n > 0; */
void SyGuS38(int n) {
  int c = 0;

  /*@
    loop invariant ((c == 0)&&(n == \at(n,Pre))) || (1
     <= c <= n) ;
    loop invariant n == \at(n,Pre);
    loop assigns c ;
  */
  while (unknown()) {
    if(c == n) {
      c = 1;
    }
    else {
      c = c + 1;
    }
  }

  /*@ assert (c == n) ==> (c >= 0); */
}
```

Fig. 9. Case 1 (SyGuS-38): a nondeterministic loop whose phase structure is exposed by symbolic execution.

APPENDIX J
CASE STUDIES

This appendix presents ten case studies that show, on concrete programs, what symbolic execution contributes to generated specifications. Across all cases, SESPEC reuses the same local symbolic context and template-routing signals, so each case emphasizes a distinct role of that context rather than repeating the full mechanism. Table XI summarizes the cases, which cover loop phases, C aliasing, recursive heap shapes, RTE safety versus functional specification, an industrial pipeline, inductive helpers, signed arithmetic, recursion, and matrix multiplication. In the figures below, ACSL annotations enclosed by $/*@\cdots*/$ are generated by SESPEC and then retained after verification or pruning.

TABLE XI
ORGANIZATION OF THE CASE STUDIES.

Case	Main lesson
1. SyGuS loop	Phase-split invariant from weak goals
2. TripleAbsMax	Path-complete C/ACSL postconditions
3. Linked list	Shape-predicate routing
4. Linear search	RTE-safety vs. functional specification
5. Industrial pipeline	Scale and verifier boundary
6. Factorial	Guarded inductive helper
7. GCD	Recursive summary with a simple measure
8. MulLoop	Sign-sensitive loop invariant
9. Ackermann	ACSL/SMT recursion boundary
10. Matrix multiply	Nested loops; verifier/SMT-timeout boundary

A. Case 1: Phase-Split Invariant from a Weak Goal

Fig. 9 isolates the benefit of symbolic execution when the assertion is too weak to guide invariant synthesis. The goal $(c = n) \Rightarrow (c \geq 0)$ can be proved by many uninformative invariants; what matters here is the loop's phase structure.

At the loop boundary, symbolic execution separates the unentered state (`c == 0`) from the inductive region (`1 <= c <= n`) and records that `n` is stable. The disjunctive template instantiates exactly that split, yielding an invariant that describes the loop behavior rather than merely satisfying the final assertion.

Case 2: path-complete postconditions for pointer-manipulating code

```
typedef struct __TripleAbsMax
{
  int abs[3];
  int tmax;
  int* ret;
} TripleAbsMax;

/*@
  requires \valid(pIp);
  requires \valid(pIp->abs+(0..2)) ;
  requires \valid(pIp->ret);
  requires \separated(pIp, pIp->ret);

  ensures pIp->ret == \old(pIp->ret);
  ensures -pIp->abs[2] <= pIp->abs[1] && pIp->abs[0] <= pIp->abs[1] && pIp->abs
       [2] < 0 && pIp->abs[1] >= 0
  && pIp->abs[0] >= 0 ==> pIp->tmax == pIp->abs[1] && *\old(pIp->ret) == pIp->
       abs[1];
  ... all 32 paths

  assigns pIp->tmax, *(pIp->ret);
*/
void TripleAbsMaxFun(TripleAbsMax *pIp)
{
  int absfx1 = pIp->abs[0];
  int absfy2 = pIp->abs[1];
  int absfz3 = pIp->abs[2];

  if (pIp->abs[0] < 0){  absfx1 = -pIp->abs[0];}

  if (pIp->abs[1] < 0){  absfy2 = -pIp->abs[1];}

  if (pIp->abs[2] < 0){  absfz3 = -pIp->abs[2];}

  if (absfx1 > absfy2){  pIp->tmax = absfx1;}
  else{ pIp->tmax = absfy2; }

  if (absfz3 > pIp->tmax){pIp->tmax = absfz3;}

  *(pIp->ret) = pIp->tmax;
}

/*@
  requires \valid(pIp);
  requires \valid(pIp->abs+(0..2));
  requires \valid(pIp->ret);
  requires \separated(pIp, pIp->ret);
*/
void main1(TripleAbsMax *pIp)
{
  pIp -> abs[0] = 1;
  pIp -> abs[1] = 2;
  pIp -> abs[2] = -3;

  TripleAbsMaxFun(pIp);

  /*@ assert pIp -> tmax == 3; */
}
```

Fig. 10. Case 2 (TripleAbsMax): a loop-free program with 32 feasible paths under the precondition shown.

B. Case 2: Path-Complete Postconditions for Pointer-Manipulating Code

Fig. 10, from our **pIp** benchmark, is the main C-specific case. The function computes the maximum absolute value of three struct-stored integers and writes it both to `pIp->tmax` and through `pIp->ret`.

Starting from the default precondition, including `\separated(pIp, pIp->ret)`, symbolic execution derives the guarded postconditions for the feasible paths.

C. Case 3: Shape-Predicate Routing for Linked Structures

Fig. 11 illustrates template routing for a linked-list traversal. The loop walks a singly linked list and returns the original head pointer, so ACSL needs an explicit heap-shape abstraction instead of only arithmetic facts.

Symbolic execution identifies `p` as the advancing list pointer and records the local update facts, while the static route recognizes the recursive type and field-access pattern. That route selects the recursive-shape template, which supplies `lseg`, `listrep`, and `list_length`; the loop template then connects the original head to the current suffix through `lseg(l, p)`. The program-structure route supplies shape

Case 3: route-selected shape predicates

```
struct SLL {
    struct SLL *tail;
    int head;
};

/*@
  predicate lseg{L}(struct SLL* x, struct SLL* y) =
    x == y || (x != y && \valid(x) && \separated(x, y)
     && lseg{L}(x->tail, y));
*/

/*@ predicate listrep(struct SLL* head) = lseg(head, \
     null); */

/*@
  logic integer list_length{L}(struct SLL* x) =
    (x == \null) ? 0 : 1 + list_length{L}(x->tail);
*/

/*@
  requires listrep(l);
  requires \valid_read(&data);
  assigns \nothing;
  ensures listrep(l);
  ensures \result == l;
*/
struct SLL * main1(struct SLL *l, int data)
{
    struct SLL *p;
    p = l;

    /*@
      loop invariant listrep(p);
      loop invariant listrep(l);
      loop invariant lseg(l, p);
      loop invariant data == \at(data,Pre);
      loop invariant l == \at(l,Pre);
      loop invariant list_length(l) >= list_length(p);
      loop assigns p;
    */
    while (p) {
      if (p->head == data) {
        /*@ assert data == \at(data,Pre); */
        /*@ assert l == \at(l,Pre); */
        return l;
      }
      p = p->tail;
    }

    return l;
}
```

Fig. 11. Case 3 (linked-list traversal): a recursive data-structure program with shape predicates.

predicates, and symbolic cursor facts anchor them to this traversal.

D. Case 4: RTE Safety vs. Functional Specification

We reuse the linear search example from Fig. 3 of Preguss [16] and run SESPEC on the same source. The point is not a head-to-head accuracy comparison: Preguss is driven by RTE-safety obligations, while SESPEC attempts to generate goal-independent functional specifications. This distinction also affects failure handling. On the Frama-C validation side, SESPEC records potential RTE reports as warnings rather than treating RTE elimination as the main target. On the QCP side, however, an RTE can still block symbolic execution; the flattened-memory default precondition covers validity, bounds, and separation facts exposed by the model, but it does not guarantee that every arithmetic or pointer-related RTE is avoided. The refinement stage can ask the LLM to strengthen preconditions to avoid such failures, but this is a supporting mechanism for functional specification generation rather than the objective itself.

Preguss: RTE-guided safety specification

```
/*@ requires \valid_read(arr+(0..len-1));
    assigns \nothing; */
int * search(int * arr, int len, int val) {
    int i = 0;
    /*@ loop assigns i; */
    while (i < len) { ... }
}
```

SESPEC: verifier-checked prefix-search specification

```
/*@ logic integer count_eq{L}(int* a, integer lo,
      integer hi, integer v) =
      lo >= hi ? 0 : count_eq(a, lo, hi - 1, v)
                   + (a[hi - 1] == v ? 1 : 0); */
/*@ requires \valid(arr + (0 .. len - 1));
    requires len >= 0;
    ensures \forall integer k; 0 <= k < len ==>
              arr[k] == \at(arr[k], Pre);
    ensures \result == \null ==>
              count_eq(arr, 0, len, val) == 0;
    ensures \result != \null ==>
              count_eq(arr, 0, \result - arr, val) == 0
              && *\result == val;
    assigns \nothing; */
int * search(int * arr, int len, int val) {
    int i = 0;
    /*@ loop invariant 0 <= i <= len;
        loop invariant \forall integer k;
            0 <= k < len ==> arr[k] == \at(arr[k], Pre);
        loop invariant count_eq(arr, 0, i, val) == 0;
        loop assigns i; loop variant len - i; */
    while (i < len) {
        if (arr[i] == val) return arr + i;
        i = i + 1;
    }
    return (int *)0;
}
```

Fig. 12. Case 4 (linear search). Preguss verifies memory-access safety; SESPEC also captures the prefix-search property and returned-pointer consequence.

Fig. 12 shows the difference in target. Preguss emits the memory-safety precondition needed to discharge `arr[i]` accesses. SESPEC instead routes the loop to a count-occurrences helper and records both return cases: null means `val` is absent, and non-null means the returned pointer dereferences to `val` with no earlier occurrence. This illustrates why RTE-safety synthesis of Preguss [16] and our functional specification synthesis should not be read as identical tasks.

E. Case 5: Industrial Pipeline Stress Test

Program structure. We applied SESPEC to a 225-line C transcription of an industrial single-star-sensor Kalman-update pipeline. The translation unit contains 14 functions, 33 call sites, and 8 loops; the entry delegates to three pipeline stages, which in turn call ten numeric and quaternion helpers (Fig. 13). This case is included to show the scale boundary, not another single function invariant pattern.

What SESPEC produced. Driven by `gpt-5.4-mini` at maximum reasoning effort (`reasoning_effort=high`) in the goal-independent configuration, SESPEC completed specification generation for 13 of the 14 functions. The assembled file contains 198 ACSL clauses, and Frama-C/WP discharges 601 of 625 verification conditions (96.2%) at a 30 s per-goal timeout. Table XII shows that increasing the timeout from 5 s to 30 s recovers only one additional goal, so the residual failures reflect hard proof obligations rather than transient budget noise.

To complement the proof-discharge rate, we also manually audited the functional content of each accepted contract. We count a postcondition as functional only when it gives a non-vacuous relation between the output or return value and the inputs; `ensures \true` receives no such credit.

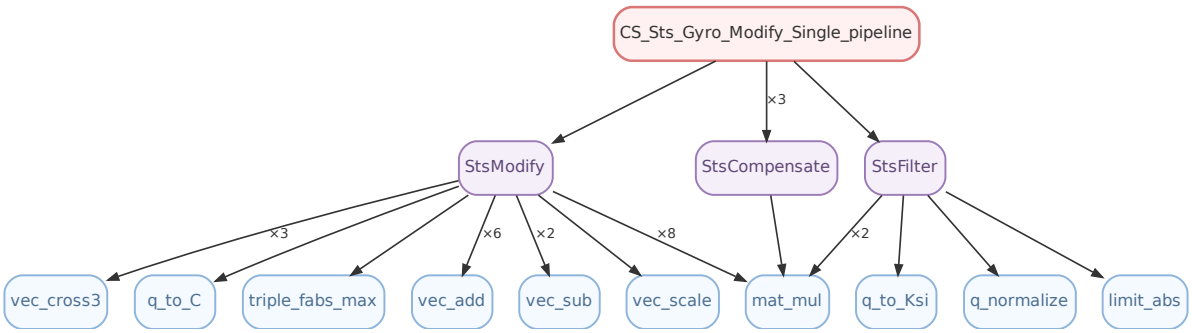


Fig. 13. Call graph of the single-star-sensor filter-update pipeline. Edge labels show call multiplicity; unlabeled edges are ×1.

TABLE XII
FRAMA-C/WP PROOF OBLIGATIONS FOR THE AEROSPACE PIPELINE.

	per-goal timeout		
Disposition	**5 s**	**10 s**	**30 s**
Total	625	625	625
Valid	600	601	601
Unknown	2	2	3
Timeout	23	22	21
Discharge rate	96.0%	96.2%	96.2%

Ten functions have functional postconditions: `limit_abs`, `q_to_C`, `q_to_Ksi`, `triple_fabs_max`, `vec_cross3`, `vec_scale`, `vec_add`, `vec_sub`, `StsCompensate`, and `StsFilter`. The other four lack a complete functional summary: `StsModify` provides only output bounds, `mat_mul` has the vacuous `ensures \true`, and the no-op helper `q_normalize` and the top-level `CS_Sts_Gyro_Modify_Single_pipeline` have no explicit accepted contract.

Boundary exposed. The functions with functional summaries are mostly arithmetic leaves whose semantics admit closed-form ACSL relations, such as `limit_abs`, `vec_cross3`, and `q_to_C`. The hard cases are delegate functions and matrix-style loops: composing multi-stage callee summaries and expressing row-major matrix products require either user-provided logic summaries or proof automation beyond the current SMT-only backend. Together, the 96.2% discharge rate and contract audit characterize both verifier acceptance and functional coverage. Concretely, the undischarged goals concentrate in the matrix-multiply leaf `mat_mul` and its callers, exactly the triple-nested product dissected in Case 10 below, where the recursive `mat_sum` over nonlinear indices makes Z3 time out and the accepted specification degrades to `ensures \true`.

F. Case 6: Guarded Inductive Helper for Factorial

Fig. 14 complements the phase-split loop case (Case 1) above: the loop may be skipped when `n < 1`, and the entered region requires the inductive relation `f == product(1, i - 1)`. Symbolic execution provides the entered/unentered guard, while the static route selects the inductive-helper template that supplies the ACSL `product` definition used by both the invariant and the postcondition.

G. Case 7: Recursive GCD Summary with a Simple Measure

Fig. 15 shows the recursion case where the current workflow succeeds cleanly. When symbolic execution reaches a recursive call, SESPEC switches to the recursion template, emits `gcd_logic` branch-by-branch, and uses `decreases a + b`. This case is useful mainly as a contrast to the Ackermann case below: GCD needs only a single-measure descent and a single-unfold style of reasoning.

Case 6: guarded factorial helper

```
int unknown();

/*@
  logic integer product(integer begin, integer end) =
    (end < begin) ? 1 : product(begin, end - 1) * end;
*/

/*@
  requires \true;
  assigns \nothing;
  ensures (n < 1) ==> \result == 1;
  ensures (n >= 1) ==> \result == product(1, n);
*/
int factorial9(int n) {
  int i = 1;
  int f = 1;

  /*@
    loop invariant (1 <= \at(n,Pre)) ==> (1 <= i <= n + 1);
    loop invariant (1 <= \at(n,Pre)) ==> (f == product(1, i - 1));
    loop invariant (!(1 <= \at(n,Pre))) ==> ((f == 1)&&(i == 1)&&(n == \at(n,
       Pre)));
    loop invariant n == \at(n,Pre);
    loop assigns f, i;
  */
  while (i <= n) {
    f = f * i;
    i = i + 1;
  }

  return f;
}

void goo9() {
  int t = factorial9(5);
  //@ assert t == 120;
}
```

Fig. 14. Case 6 (factorial): a guarded loop invariant with a recursive `product` helper.

H. Case 8: Signed Multiplication Loop

Fig. 16 computes $a \cdot b$ by addition or subtraction depending on the sign of `b`. The negative branch requires the asymmetric invariant `res == -i * a`; reusing the positive-branch form would make the functional postcondition fail. In our run, SESPEC retains `ensures \result == a * b` because the branch condition selects the correct invariant template.

Case 7: recursive GCD summary

```
/*@
  logic integer gcd_logic(integer a, integer b) =
    (a == 0) ? b :
    (b == 0) ? a :
    (a == b) ? a :
    (a > b) ? gcd_logic(a - b, b) :
              gcd_logic(a, b - a);
*/

/*@
requires a >= 0 && b >= 0;
decreases a + b;
assigns \nothing;
ensures \result == gcd_logic(a, b);
*/
int gcd(int a, int b)
{
    if (a == 0)
       return b;

    if (b == 0)
       return a;

    if (a == b)
        return a;

    if (a > b)
        return gcd(a-b, b);
    return gcd(a, b-a);
}

int main2()
{
    int a = 98, b = 56;
    int c = gcd(a, b);
    //@ assert c == 14;
    return 0;
}
```

Fig. 15. Case 7 (GCD): a recursive logic summary with a single arithmetic termination measure.

Case 8: signed multiplication loop

```
int foo57(int a, int b);

/*@
  requires \true;
  assigns \nothing;
  ensures \result == a * b;
*/
int foo57(int a, int b) {
    int res = 0;
    if (b >= 0) {
        /*@
          loop invariant 0 <= i <= b;
          loop invariant res == i * a;
          loop assigns i, res;
          loop variant b - i;
        */
        for (int i = 0; i < b; i++) {
            res = res + a;
        }
    } else {
        /*@
          loop invariant 0 <= i <= -b;
          loop invariant res == -i * a;
          loop assigns i, res;
          loop variant -b - i;
        */
        for (int i = 0; i < -b; i++) {
            res = res - a;
        }
    }
    return res;
}
```

Fig. 16. Case 8 (MulLoop) from **SpecGenBench**.

I. Case 9: Ackermann Recursion Beyond ACSL/SMT Automation

Fig. 17 marks a boundary rather than a prompt failure. The generated helper follows the C branch structure, but Ackermann needs a lexicographic termination argument and nested induction that are not handled well by the current ACSL/SMT stack. A scalar `decreases m * 1000 + n` is not a sound substitute for the true lexicographic descent once `n` grows, and the functional postcondition would require proof support beyond the single-unfold reasoning that succeeds for GCD.

J. Case 10: Matrix Multiplication Beyond Per-Loop Symbolic Context

Fig. 18 is the multi-loop boundary that Reviewer 2's failure-case question targets. SESPEC emits the intended quantified postcondition and the matching outer-loop invariants, while the innermost `s == mat_sum(...,k)` invariant is discharged by one-step induction. The remaining quantified obligations time out in Z3 because they combine recursive logic unfolding, havoc-updated array memory, and non-linear index arithmetic; refinement therefore strips the unproved clauses and the accepted specification collapses to verifier-valid but vacuous `ensures \true`. This case illustrates a proof-automation limit rather than a generation failure.

APPENDIX K
DISCUSSION: LIMITATIONS AND OPEN CHALLENGES

Scope and guarantees. SESPEC targets self-contained programs. Symbolic execution stops at external or system calls, including allocation through `malloc`/`free`, and passes the affected segment to LLM fallback. The strongest-postcondition guarantee in Section III and Proposition 1 applies to covered loop-free segments under path-complete, precise symbolic execution in the flattened memory model. Loops, recursive functions, and recursive data structures receive verifier-checked, best-effort specifications. A discharged specification may still omit functional relations that require unavailable symbolic states, heap abstractions, auxiliary lemmas, or reasoning beyond

Case 9: Ackermann recursion boundary

```
int fool(int m, int n);

/*@
  logic integer ackermann(integer m, integer n) =
    (m == 0) ? n + 1 :
    (n == 0) ? ackermann(m - 1, 1) :
               ackermann(m - 1, ackermann(m, n - 1));
*/

/*@
  requires m >= 0 && n >= 0;
  decreases m * 1000 + n;
  assigns \nothing;
  ensures \result == ackermann(m, n);
*/
int fool(int m, int n) {
    if (m == 0) {
        return n + 1;
    }
    if (n == 0) {
        return fool(m - 1, 1);
    }
    return fool(m - 1, fool(m, n - 1));
}
```

Fig. 17. Case 9 (Ackermann). The candidate specification is syntactically clean but not discharged by Frama-C/WP under the current ACSL/SMT backend.

the configured backend. Allocation modeling and reusable library summaries are future extensions.

Scalability of symbolic execution. Symbolic execution remains difficult to scale to large programs: path explosion and the cost of carrying full symbolic stores both limit it in practice. Many symbolic executors mitigate this: separation logic and shape analysis let them reason about a class of heap-manipulating programs and recursive data structures such as lists and trees [8], while path abstraction and state merging curb path explosion [40], [41]. We deliberately choose a basic symbolic executor instead, with the aim of obtaining precise symbolic states reusable for specification synthesis rather than scaling to larger programs. For what this basic executor cannot cover, we fully leverage the LLM: when the executor reaches a boundary, the framework asks the LLM to continue *in symbolic-execution order*, carrying the latest symbolic state forward and proposing a postcondition or loop invariant consistent with it. This yields stronger specifications than an unguided prompt, but it does not make LLM-generated summaries a sound replacement for symbolic execution beyond the supported fragment.

Verifier-backend ceiling. Two upstream limits cap what can be accepted even when the intended specification is semantically correct. First, some behaviors are cumbersome or unsupported in the ACSL fragment we use, such as lexicographic measures, higher-order quantified properties, and separation-logic-style abstractions over inductive heaps. Even a singly linked-list traversal may require user-defined `lseg`/`listrep` helpers that must be both faithful and SMT-friendly. Second, the ATP backend used by Frama-C/WP cannot reliably automate all quantified, inductive, and memory-heavy obligations generated by these specifications. *Timeout* events observed in our experiments are dominated by loop-invariant obligations, especially preservation and establishment goals over arrays or heap-backed arrays. WP often translates these obligations into quantified formulas over typed-memory stores, address shifts, integer casts, and occasionally recursive logic helpers such as counting functions, so even plausible invariants can exceed the configured per-query solver budget. *Unknown* and *Failed* events observed in our experiments mostly occur in verifier/modeling regions where the backend returns no definitive counterexample: whether the remaining obligations come from specification errors or verifier/modeling limitations remains unresolved. Table XIII counts the logged events underlying this discussion.

Case 10: matrix-multiplication proof boundary

```
/*@
  logic integer mat_sum(int *A, int *B, integer ca_rb, integer cb,
                        integer i, integer j, integer k) =
    k <= 0 ? 0 :
    mat_sum(A, B, ca_rb, cb, i, j, k - 1) +
    A[i * ca_rb + (k - 1)] * B[(k - 1) * cb + j];
*/

/*@
  requires \valid(out + (0 .. ra*cb - 1));
  requires \valid(A   + (0 .. ra*ca_rb - 1));
  requires \valid(B   + (0 .. ca_rb*cb - 1));
  requires \separated(out + (0..ra*cb-1), A + (0..ra*ca_rb-1));
  requires \separated(out + (0..ra*cb-1), B + (0..ca_rb*cb-1));
  // default precondition also bounds every cell to [0,100] and each
  // dimension product to (0,100), keeping the arithmetic overflow-free
  assigns out[0 .. ra*cb - 1];
  ensures \forall integer i, j; 0 <= i < ra && 0 <= j < cb ==>
      out[i*cb + j] == mat_sum(A, B, ca_rb, cb, i, j, ca_rb); // Z3 TIMEOUT (10
       s)
*/
void mat_mul(int *out, int *A, int *B, int ra, int ca_rb, int cb) {
  /*@
    loop invariant 0 <= i <= ra;
    loop invariant \forall integer ii, jj; 0 <= ii < i && 0 <= jj < cb ==>
        out[ii*cb + jj] == mat_sum(A, B, ca_rb, cb, ii, jj, ca_rb); // Z3
     TIMEOUT (10s)
    loop invariant \forall integer p; i*cb <= p < ra*cb ==> out[p] == \at(out[p
      ],Pre); // Z3 TIMEOUT
    loop assigns i, out[0 .. ra*cb - 1];
  */
  for (int i = 0; i < ra; i++) {
    /*@
      loop invariant 0 <= j <= cb;
      loop invariant \forall integer jj; 0 <= jj < j ==>
          out[i*cb + jj] == mat_sum(A, B, ca_rb, cb, i, jj, ca_rb); // Z3
       TIMEOUT (10s)
      loop assigns j, out[i*cb .. i*cb + cb - 1];
    */
    for (int j = 0; j < cb; j++) {
      int s = 0;
      /*@
        loop invariant 0 <= k <= ca_rb;
        loop invariant s == mat_sum(A, B, ca_rb, cb, i, j, k); // DISCHARGED (
       one-step induction)
        loop assigns k, s;
      */
      for (int k = 0; k < ca_rb; k++)
        s = s + A[i*ca_rb + k] * B[k*cb + j];
      out[i*cb + j] = s;
    }
  }
}
```

Fig. 18. Case 10 (matrix multiplication, $ra \times ca_rb$ by $ca_rb \times cb$). SESPEC generates the correct quantified specification, but Frama-C/WP→Z3 times out on the outer invariants and postcondition, so the accepted specification degrades to `ensures \true`.

TABLE XIII
VERIFIER LOG EVENTS FOR THE `GPT-5.4-MINI` SESPEC-400 RERUN.

Prover outcome	Loop Inv.	Post Cond.	Assigns	Other VC	Events
Syntax	—	—	—	—	2,315
Timeout	2,590	697	1,014	69	4,370
Unknown	62	4	111	0	177
Failed	27	22	0	0	49

Together with the program-feature breakdown in Fig. 5, this identifies *the SMT solving capacity of the ATP backend* as the dominant limit on our validation pass rate. Migrating to an interactive theorem prover can make residual obligations dischargeable by allowing explicit proof scripts, and in our manual inspection the remaining obligations become reachable once the LLM writes such scripts under human guidance. This does not eliminate the scalability problem: it shifts the bottleneck from ATP automation to proof construction and proof search. We therefore view ITP integration as a promising future direction for SESPEC, especially as LLMs become better at generating and searching proof scripts, but not as an already solved route to large-scale verification.

APPENDIX L
THREATS TO VALIDITY

We discuss threats to validity along the standard construct, internal, and external dimensions.

Construct validity. Our metrics — syntactic correctness (*Syn.*), validity under Frama-C/WP (*Val.*), and goal accuracy (*Acc.*) — measure proof acceptance under a specific verifier configuration, not semantic adequacy in absolute terms. A verifier-checked specification is not necessarily as informative as a hand-written one, and an unchecked one is not necessarily semantically wrong: it may be unprovable under the ACSL fragment, memory model, or ATP backend discussed in Appendix K. Quality is also hard to compare when two valid specifications differ in strength, readability, framing precision, or coverage. To partially address this, the experiment in Section VI adds an LLM-as-judge pairwise strength comparison on the both-valid intersection. This comparison is itself approximate: the judge is an LLM, may weight informativeness differently from a human

expert, and offers no complete oracle for specification quality. Reported gains should therefore be read as evidence under the chosen metrics, rather than as a complete characterization of specification quality. Indeed, how to formally evaluate the quality of a specification beyond mere verifier acceptance remains an open problem, as recent efforts to benchmark and assess LLM-generated formal specifications make clear [37], [42].

Internal validity: LLM nondeterminism and Pass@k. LLM completions are stochastic. We fix the sampling temperature at 0.7 throughout, with OpenAI's `reasoning_effort=low` on the reasoning-tier models (`gpt-5.4-mini`, `gpt-5`); the sole exception is the industrial case study in Appendix J-E, which uses `reasoning_effort=high`. Because single-shot results vary across runs, we report Pass@1, Pass@3, and Pass@5 separately for the goal-dependent benchmarks (Table VIII); the goal-independent experiment in Section VI reports Pass@1 only, as its per-attempt cost is substantially higher. The Pass@1→Pass@5 gap recovers some hard cases on **OOPSLA**, **Frama-C**, and **list-S**, but is already marginal by Pass@3 on **NLA**, **numer-S**, and **pIp**. These rates describe the studied sampling configuration and may vary across independent runs.

Internal validity: data leakage. Bias and data leakage from the LLM's pretraining corpus are a concern, especially in program verification, where domain-specific data are scarce. Training-based methods and benchmark-specific retrieval pipelines are most vulnerable, since benchmark-adjacent corpora, curated examples, or synthetically generated programs may implicitly encode benchmark-specific patterns. SESPEC mitigates this by avoiding benchmark-specific training and dataset-dependent demonstrations: it does not fine-tune the model or train auxiliary components on the evaluation benchmarks, and its fixed prompts draw runtime content from the target program's symbolic facts and static features rather than from labels or expected specifications. Pretraining exposure to benchmark-related material remains possible. The reported gains provide evidence for the effectiveness of symbolic-execution-guided prompting under these experimental settings.

External validity: benchmark and baseline selection. The evaluation covers small and medium-size verification tasks from the AUTOSPEC suite, LIPuS, and CLAUSE2INV, plus our industry-derived **pIp** set and 400-case **SESPEC-400**. The **SV-COMP** subset is the AUTOSPEC-curated 21-program slice. These results characterize self-contained programs with the studied control and data structures; large industrial codebases and extensive external-library use remain outside the evaluated scope. Scaling functional verification further is constrained by the underlying proof problem. The first wall is the reach of automated theorem provers (ATPs), which time out or return *Unknown* on the nonlinear, quantified, and recursively defined obligations that realistic code induces, as our own Frama-C/WP runs repeatedly show (Appendix K). Migrating these obligations to an interactive theorem prover (ITP) changes this boundary rather than removing it: an ITP offers a richer proof surface for obligations that ATP backends fail to discharge, but it also shifts the burden to proof construction and proof search. Recent LLM-driven ITP efforts therefore remain evaluated on relatively small programs, while also suggesting that this route may become more scalable as LLM proof-script generation and proof search improve [37], [38], [42], [43].

External validity: memory model and verifier soundness. The verifier-backend completeness limits are discussed in Appendix K; here we isolate the soundness assumptions behind the reported verification results. We use Frama-C/WP's *Typed* memory model, which is known to be unsound for several C idioms, including pointer-to-pointer casts that violate effective types, accesses that reinterpret memory between different scalar types, reads or writes through unions that mix the active variant, and overlapping or misaligned accesses to the same region. The evaluated benchmarks use none of them: **SyGuS**, **OOPSLA**, **NLA**, **numer-S**, and **SV-COMP** are numerical loops without unions or type punning, while **Frama-C**, **pIp**, and **list-S** use structs, pointers, and singly linked lists but do not cast pointers across incompatible types or access overlapping union variants. Dynamic allocation through `malloc` is outside the modeling scope. Programs outside these memory-model assumptions are outside the soundness envelope of this evaluation.